

30分で完走

完全未経験歓迎

GitHub入門講座

～ はじめてのPull Request & 草を生やそう ～

学生AI/データサイエンスコミュニティ **Cypher**

この30分のゴール



あなたのPull Requestがmergeされ、
GitHubのプロフィールに「草」が1つ生えている。

 草（くさ）とは？

GitHubで開発活動（コミットなど）をした日にプロフィールに付く**緑のマス目**のこと。

 Pull Request（PR）とは？

「自分の変更を本番に取り込んでください！」という**公式のお願い投稿**のこと。

最初に、いちばん大事な約束

🛡 あなたのPCもリポジトリも壊れません

Gitは「歴史を記録する道具」です。失敗してもいつでも過去に戻せます。

- ✖ エラーが出た？ ➡ 正常です！ 講師も毎日10回はエラーを出します。
- ? 分からなくなった？ ➡ 即、手を挙げてください！ TAと講師が助けます。
- 🖥 黒い画面が怖い？ ➡ 大丈夫、今日打つコマンドは数行だけです。

💡 エラーメッセージは「あなたを怒る文章」ではなく**「次へのヒント」**です。

今日のタイムスケジュール（30分）

時間	パート	内容
00-03分	導入	ゴール確認・事前課題セルフチェック
03-07分	Part 1	ターミナルってなに？ <code>pwd</code> / <code>cd</code> / <code>ls</code>
07-12分	Part 2	Git vs GitHub & 荷物でわかる4領域モデル
12-24分	Part 3	【ハンズオン】自己紹介を作ってPRを出そう！
24-27分	Part 4	プロフィールの草を確認 & 生えない罫の対策
27-30分	Part 5	まとめ & AI時代におけるGitの付き合い方

 **GUI派の方へ**：全手順をVS Codeのマウス操作で行う手順書もあります！

事前課題 クイックチェック

手元のターミナル（Windowsは Git Bash）で確認してみましょう！

#	確認項目	打つコマンド	期待される結果
1	Gitが入っている	<code>git --version</code>	<code>git version 2.xx.x</code>
2	名前が登録されている	<code>git config --global user.name</code>	あなたのGitHub ID
3	メールが登録されている	<code>git config --global user.email</code>	GitHubの登録メール
4	GitHubログイン済み	<code>gh auth status</code>	✓ Logged in to github.com

⚠ まだの項目がある人、赤字のエラーが出た人は**すぐに手を挙げてください！**

Part 1 (03-07分)

ターミナルってなに？

黒い画面は「文字でおしゃべりする道具」

ターミナル（黒い画面）の正体

普段のマウス操作と、やっていることは**まったく同じ**です！

GUI（マウス操作）

- フォルダをダブルクリックして開く
- 右クリック  「新規作成」
- 目で見て直感的に動かせる

CUI / ターミナル

- `cd folder` と指示して移動する
- `touch file` と指示して作る
- **自動化やAIとの連携が得意！**

たとえば：GUIが「お店で指差し注文」なら、ターミナルは「LINEでテキスト注文」。

ターミナルで一番大事なこと：「今どこにいるか」

ターミナルは**「今いる場所（カレントディレクトリ）」**を基準にして動きます。

カレントディレクトリ（Current Directory）＝ ターミナルが今開いているフォルダのこと

```
/Users/taro/  
├─ Desktop/      ← デスクトップ  
├─ Downloads/    ← ダウンロード  
└─ Documents/    ← 書類
```

- 今 **Desktop** にいるのか？ それとも **Downloads** にいるのか？
- **場所を見失ったら、まず「今どこ？」を確認する**のが鉄則です！

覚えるコマンドは3つだけ！

`pwd`（今どこ？）

Print **W**orking **D**irectory

今いるフォルダの場所（パス）を画面に表示します。

`ls`（何がある？）

List

今いるフォルダの中にあるファイルやフォルダの一覧を表示します。

`cd フォルダ名`（そこへ移動！）

Change **D**irectory

指定したフォルダの中に移動します。（例：`cd Desktop` でデスクトップへ移動）

※ `cd ..`（ドット2つ）で「1つ上の階層に戻る」

ターミナル ミニ体験

実際に打ってみましょう！（1行打ったらEnterキー）

```
pwd
```

```
/Users/taro    #（あなたの現在地が表示されます）
```

```
ls
```

```
Desktop  Documents  Downloads  #（フォルダの一覧が出ます）
```

```
cd Desktop
```

```
pwd
```

```
/Users/taro/Desktop  #（デスクトップに移動できました！）
```

Part 2 (07-12分)

Git vs GitHub & 4領域モデル

荷物の発送で仕組みを一発理解する

Git と GitHub は「別物」です

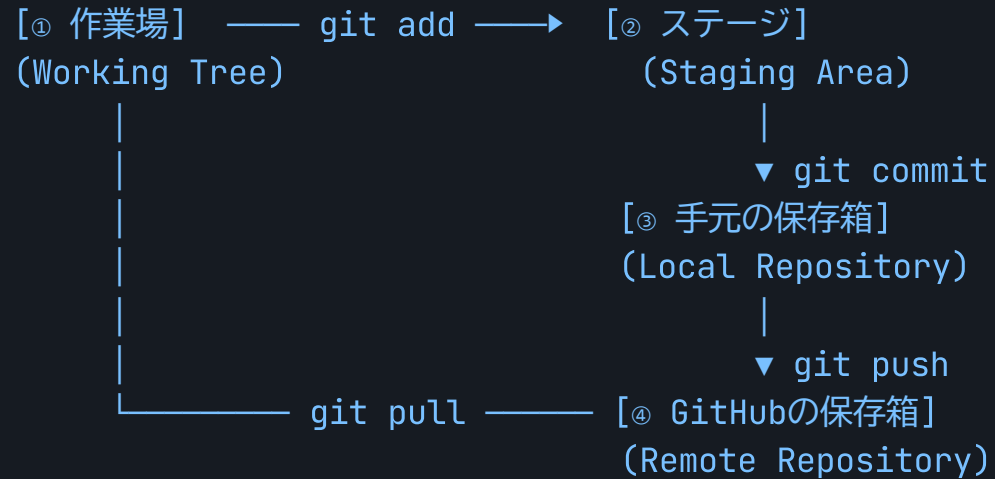
名前は似ていますが、役割がまったく違います！

	Git（ギット）	GitHub（ギットハブ）
正体	あなたのPCに入る アプリ・道具	ネット上の Webサービス
場所	手元のパソコンの中	クラウド（インターネット）
役割	ファイルの変更履歴を 記録 する	履歴を 共有 してみんなで開発する
ネット	不要 （オフラインで動く！）	必要

たとえば：Git = **カメラ（撮影する道具）** | GitHub = **Instagram（写真を投稿・共有する場所）**

Gitの「4領域モデル」

Gitの世界には **4つの場所** があります。変更はここを順番に移動します。



①～③はあなたのPCの中。④だけがインターネット上です。

荷物の発送でたとえる「4領域」

1 作業場 (Working Tree)

部屋で荷物を広げて作業している状態。

（ファイルを編集しただけ）

2 ステージ (Staging Area)

「送る物」を選んでダンボールに入れた状態。

（`git add` で選別）

3 ローカル (Local Repository)

ダンボールにガムテープを貼り、封をした状態。

（`git commit` で手元に記録確定）

4 リモート (GitHub)

郵便局に持っていき、相手に届いた状態！

（`git push` で世界へ公開）

よくある疑問：「なぜステージがあるの？」

「ファイルを編集したら、そのまま保存（commit）じゃダメなの？」

💡 理由：関係ない変更を混ぜないため！

例えば、同時に3つのファイルを編集したとします：

- `login.py`（ログイン機能の追加） ➡ 今回のコミットに含めたい！
- `test_memo.txt`（個人的な落書きメモ） ➡ 人に見せたくない…

`git add login.py` とすることで、「送りたい物だけ」を綺麗に選んで箱詰めできます。

Part 3 (12-24分)

ハンズオン本編！

自己紹介ファイルを作って Pull Request を出そう

これからやるハンズオンの流れ（GitHub Flow）

実務のエンジニアが毎日やっている**「王道の開発手順」**を体験します！

1. 📁 **clone** ➡ プロジェクトを自分のPCに持ってくる
2. 🌿 **branch** ➡ 自分専用の作業ブランチ（枝）を作る
3. 📝 **編集** ➡ `members/あなたのID.md` を作成して自己紹介を書く
4. 📦 **add** ➡ 変更をステージ（発送箱）に入れる
5. 🔒 **commit** ➡ 変更を手元の記録として確定する
6. 🚀 **push** ➡ GitHubに送り届ける
7. 📬 **PR作成** ➡ 「取り込んでください！」とPRを出す
8. 🎉 **Merge** ➡ 講師が承認して合体！

Step 1. リポジトリを手元に持ってくる

CLI **Desktop** に移動して、リポジトリを丸ごと複製（clone）します。

```
cd Desktop  
git clone https://github.com/guousuideo/git-lec.git
```

VS Code 「クローン」 ボタンを押し、上のURLを貼り付けます。

期待される結果： デスクトップに **git-lec** フォルダが新しく作られます！

```
cd git-lec
```

⚠ 超重要： 必ず **cd git-lec** でフォルダの中に入ってください！

Step 2. 自分専用の作業ブランチを作る

みんなで同じ本番（`main`）を直接いじると事故が起きます。

安全のため**「自分専用の枝分かれ（ブランチ）」**を作って作業します。

CLI 自分のID名でブランチを作成して切り替えます：

```
git switch -c add-member-あなたのID
```

(例： `git switch -c add-member-taro-yamada`)

VS Code 左下の `main` をクリック ➡ 「新しい分岐の作成」

期待される結果： `Switched to a new branch 'add-member-...'` と出ればOK！

Step 3. 自己紹介ファイルを作る

`members/` フォルダの中に、`あなたのGitHub ID.md` を作成します！

📁 ファイルの場所

`members/taro-yamada.md`

（※拡張子は `.md` です）

📝 中に書く内容（自由）

自己紹介

- 名前: 山田 太郎
- 所属: 〇〇大学 / Cypher
- 一言: GitHubマスターになります！

💡 VS Codeのエクスプローラーから `members` フォルダを右クリックして「新規ファイル」で作ると簡単です！

Step 4. 今の状態を確認する

今、Gitから見てファイルがどうなっているか確認してみましょう！

```
git status
```

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  members/taro-yamada.md   # ← 赤文字で表示されます！
```

- **赤文字（Untracked）** = 「部屋に新しい物があるけど、まだ発送箱（ステージ）には入っていませんよ」という合図です。

Step 5. 送信箱に入れる（add）

作成した自己紹介ファイルを、ステージ（送信箱）に乗せます。

CLI

```
git add members/あなたのID.md
```

(または `git add .` で今のフォルダの全変更をステージ)

VS Code ソース管理パネルで、ファイル名の横の「+」ボタンをクリック

もう一度 `git status` を打ってみましょう：

```
Changes to be committed:
  new file:   members/taro-yamada.md   # ← 緑文字に変わった！
```

Step 6. 記録を確定して封をする（commit）

「何を変更したか」のメッセージを添えて、手元に記録します。

CLI

```
git commit -m "docs: add taro-yamada to members"
```

VS Code メッセージ欄に `docs: add ...` と入力して「コミット」ボタン

```
[add-member-taro-yamada a1b2c3d] docs: add taro-yamada to members  
1 file changed, 5 insertions(+)  
create mode 100644 members/taro-yamada.md
```

これで「PC内の保存箱」にしっかり記録されました！🎉

Step 7. GitHubへ送り出す（push）

手元に作った記録を、インターネット上のGitHubへアップロードします！

CLI

```
git push -u origin add-member-あなたのID
```

VS Code

「**ブランチの発行**」 または 「**変更の同期**」 ボタン

```
To https://github.com/guousuideo/git-1ec.git
* [new branch]      add-member-taro-yamada -> add-member-taro-yamada
Branch 'add-member-taro-yamada' set up to track remote branch.
```

GitHubにあなたのブランチが届きました！ 🚀

Step 8. Pull Request (PR) を作成する

GitHubの画面（ブラウザ）を開きましょう！

1. リポジトリ（ <https://github.com/guousuideo/git-1ec> ）を開く
2. 上部に黄色いバー **「Compare & pull request」** ボタンが出るのでクリック！

```
add-member-taro-yamada had recent pushes 1 minute ago |  
[ Compare & pull request ] |
```

※ボタンが出ない場合：「Pull requests」タブ ➡ 「New pull request」 ➡ 自分のブランチを選択

Step 9. PRの内容を書いて送信！

1. タイトル

`docs: add taro-yamada to members`

のように分かりやすく書きます。

2. 本文

テンプレのチェックボックスを埋めます：

- ☒ 自己紹介を作成した

緑色の「**Create pull request**」をクリック！   送信完了！

Step 10. レビュー & Merge ! 🎉

 講師がその場で全員のPRをMerge（合体）します！

GitHub上のPR画面が「**Merged (紫色のアイコン)**」に変わる瞬間を見届けましょう！

紫色の **Merged** マークがついたら、**あなたの一連の作業は大成功です！** 🎊

Part 4 (24-27分)

草を確認しよう！

プロフィールに緑色のマス目はついたかな？

自分のプロフィールを見に行こう！

ブラウザで自分のプロフィールページを開いてみましょう：

<https://github.com/あなたのID>



今日のマス目に色がついて「草」が生えています！

「草が生える」＝あなたが世界に向けてオープンな開発活動を一步踏み出した証拠です！

もし草が生えていなかったら？「3大罨」

「PRはマージされたのに草が生えていない！」という場合のチェックリスト：

罨① メールアドレスの不一致（90%はこれ）

`git config user.email` のメアドと、GitHubに登録されているメアドが1文字でも違うと草になりません。

罨② まだマージされていない

ブランチにコミットしただけでは草になりません。`main` にマージされて初めて反映されます。

罨③ 自分のFork（コピーリポジトリ）で作業してしまった

Fork先のコミットは、元のリポジトリにPRを出してマージされるまで草カウントされません。

罣①（メアド不一致）の治し方

ターミナルで設定を修正し、GitHub側にもメールを追加すれば解決します！

```
# 1. 今の設定を確認
git config --global user.email

# 2. GitHubに登録しているメアドに上書き設定
git config --global user.email "your-correct-email@example.com"
```

💡 GitHubの [Settings] ➡ [Emails] に、PC側で設定していたメールアドレスを追加登録することでも、過去のコミットが自分のものとして紐づけられて草が生えます！

Part 5 (27-30分)

まとめ & AI時代のGit

AIがコマンドを打つ時代に、人間は何をするのか？

今日できるようになったこと 🎓

たった30分で、実務のエンジニアと同じスキルを習得しました！

- ☒ **ターミナルの基本**（`pwd`，`cd`，`ls` で現在地を迷わない）
- ☒ **Gitの4領域モデル**（作業場 ➡ ステージ ➡ ローカル ➡ リモート）
- ☒ **GitHub Flowの実践**（clone ➡ branch ➡ commit ➡ push ➡ PR）
- ☒ **草を生やす仕組み**（メール認証・マージ条件）

番外編1：コンフリクト（衝突）と元に戻す方法

同じファイルの同じ行を2人が同時に編集すると**「コンフリクト（衝突）」**が起きます。

✖️ コンフリクトは怖くない

Gitが「どっちを採用する？」と親切に聞いてくれている状態です。VS Codeならボタン1つで選べます。

⏮️ やり直す道具（revert）

間違えてマージした時は `git revert` で「打ち消すコミット」を作れば安全に戻せます。

詳しくは 📄 `docs/05_extra_conflict_revert.md` と `conflict-playground/` をチェック！

番外編2：AIエージェント時代のGit操作

現代は、Claude Code や GitHub Copilot などのAIが

自然言語の指示だけで `add` / `commit` / `push` / PR作成 まで自動実行できる時代です。

💬 人間「自己紹介を追加してPRを作っておいて」

🤖 AI「変更をステージングし、コミットしてPR #12 を作成しました！」

「じゃあ、人間がGitを勉強する意味はなくなったの？」

➡ 逆です。これまで以上にGitの理解が必要になっています。

なぜAI時代に「4領域」と「diff」を学ぶのか？

⚠️ AIも間違える

- パスワードや `.env` をコミットしてしまう
- 消してはいけないコードを消してしまう
- 勝手に `push --force` してしまう

🛡️ 人間の最後の責任

- AIが出した**差分（diff）**を**自分の目で読む**
- 「今どの領域に何があるか」を把握する
- 問題がないことを確認して**承認（Approve）**する

「AIに任せる範囲が増えるほど、何が起きたかを読み解く力が武器になる」

これからのおすすめアクション

1 もう1回PRを出してみる

`members/あなたのID.md` を編集して、趣味や特技を追記してPRを出してみよう！2回目は驚くほどスムーズです。

2 自分のリポジトリを作る

大学のレポートやプログラミングの学習記録をGitHubで管理してみよう！

3 Cypherのプロジェクトに参加する

コミュニティの開発プロジェクトで、仲間と一緒にチーム開発を実践しよう！

質問・困ったときのサポート

今日の講座が終わった後も、いつでも質問大歓迎です！

Cypher Discord

`#git-lec` チャンネルでいつでもメンションしてください！

GitHub Issues

このリポジトリの「Issues」から質問を立てるのも大歓迎！立派な練習になります。

Happy Hacking! 🌿

GitHubの世界へようこそ！

学生AI/データサイエンスコミュニティ **Cypher**